

Class 05: Data Visualization with GGPLOT

Joshua Chun (PID: A17812847)

Table of contents

Background	1
Add some custom features	4
Gene Expression Figure	6
Going further	18

Background

There are many graphics systems in R for making plots and figures. These include so-called “*base R*” *graphics* like this `plot()` function and add on packages like **ggplot2**.

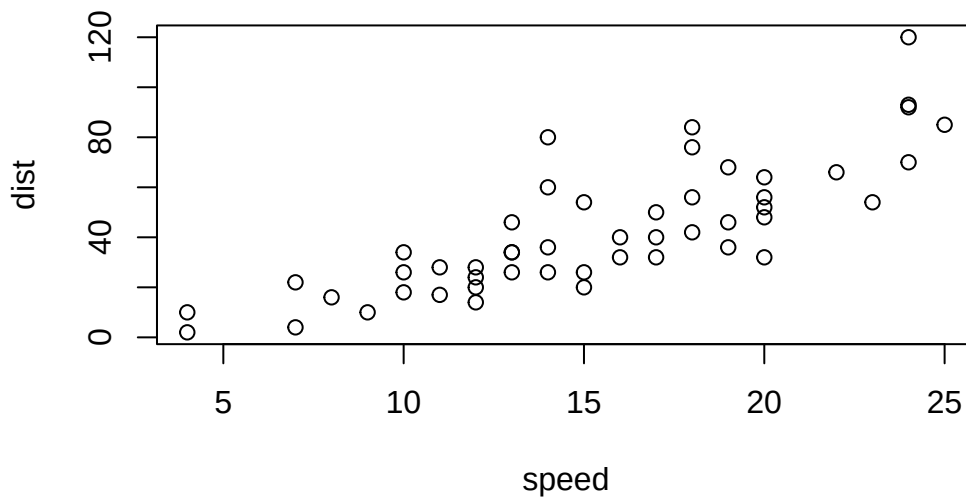
Let’s compare how we make a simple figure with these two systems:

We can use the in-built `cars` dataset:

```
head(cars)
```

```
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
```

```
plot(cars)
```



Before I can use `ggplot2` I need to install it on my computer. To do this we can use the function `install.packages("ggplot2")`

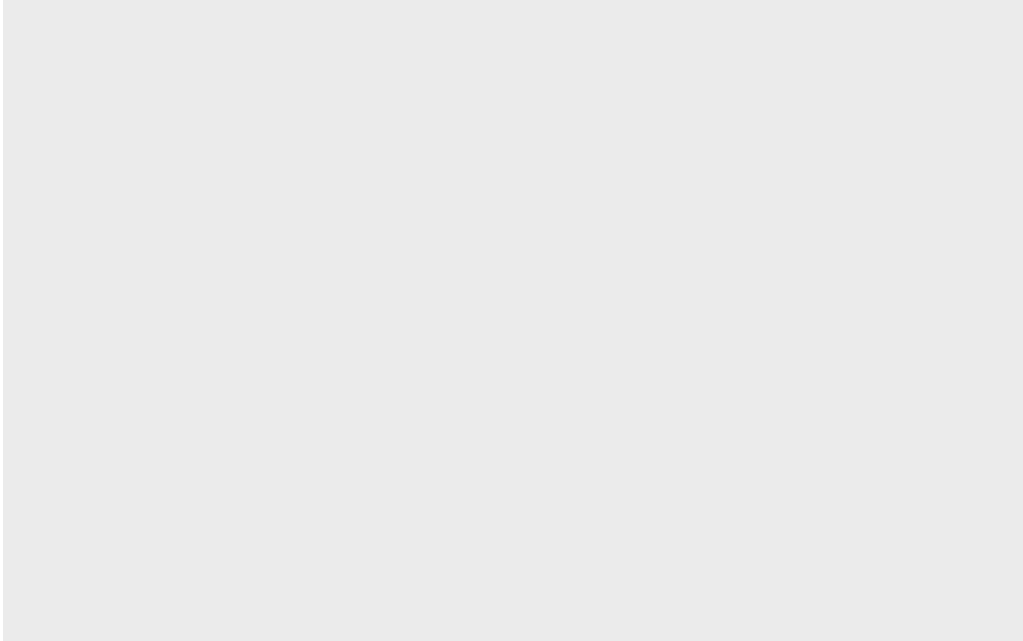
N.B. We never run `install.packages()` in our quarto doc (we run it once only in our console) as it would re-install the package every time we render our quarto report.

Once installed we need to load up the package into our R brain:

```
library(ggplot2)
```

The main function in **ggplot2** package is called `ggplot()`

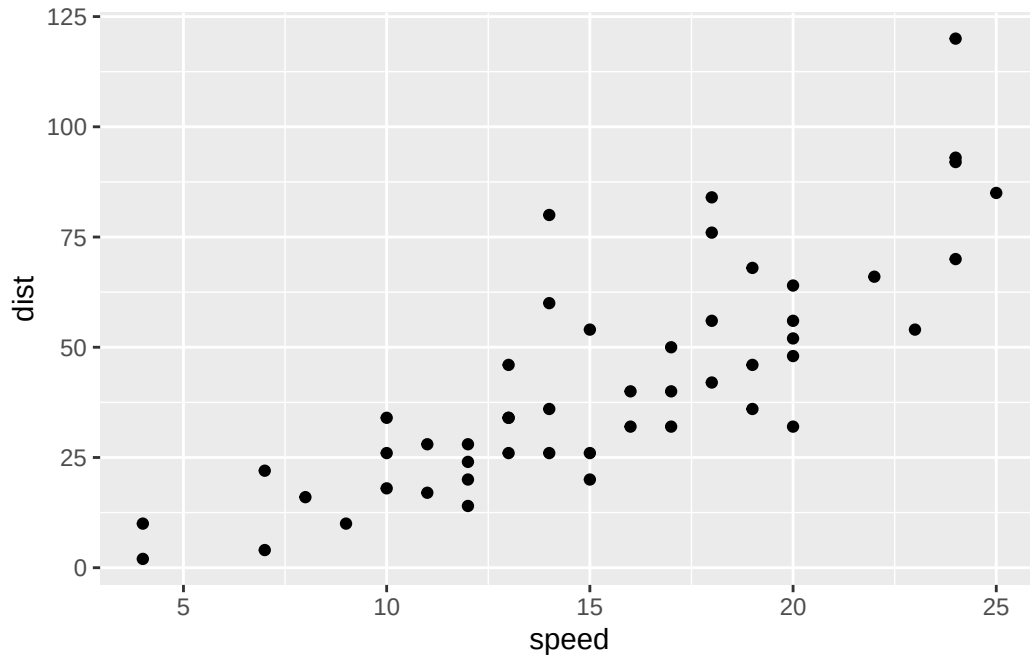
```
ggplot(cars)
```



Every ggplot has at least 3 layers:

- the **data** (a data.frame of the stuff we want to plot)
- the **aesthetics** (how the data maps to the plot)
- the **geom** layer (how you want the plot drawn, e.g. points, lines, etc.)

```
ggplot(cars) +  
  aes(x=speed, y=dist) +  
  geom_point()
```

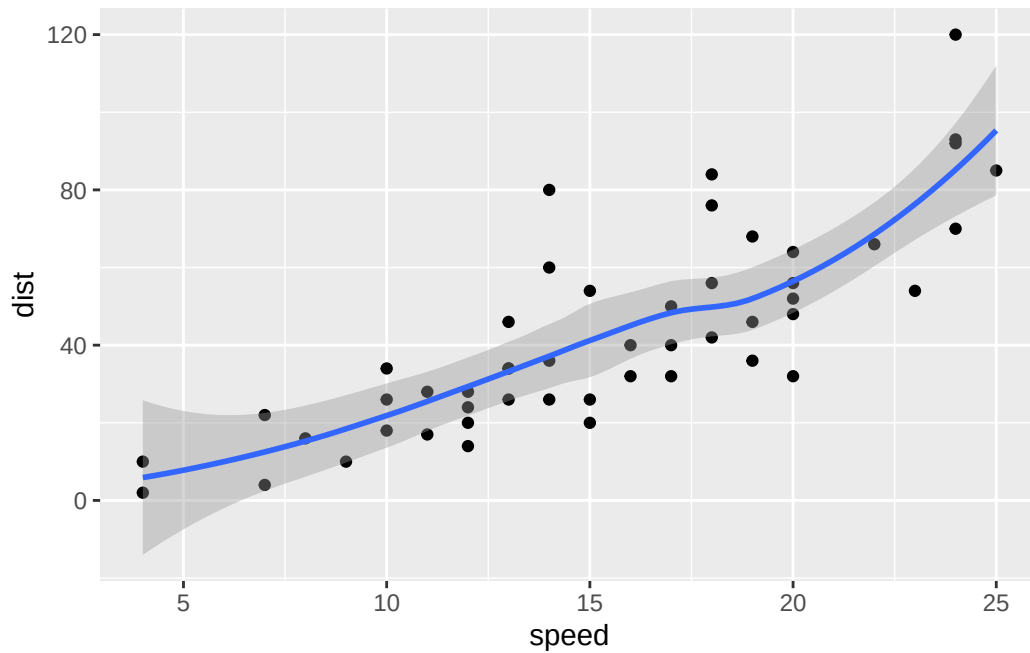


Add some custom features

Let's add a trend line that shows the relationship between speed and distance.

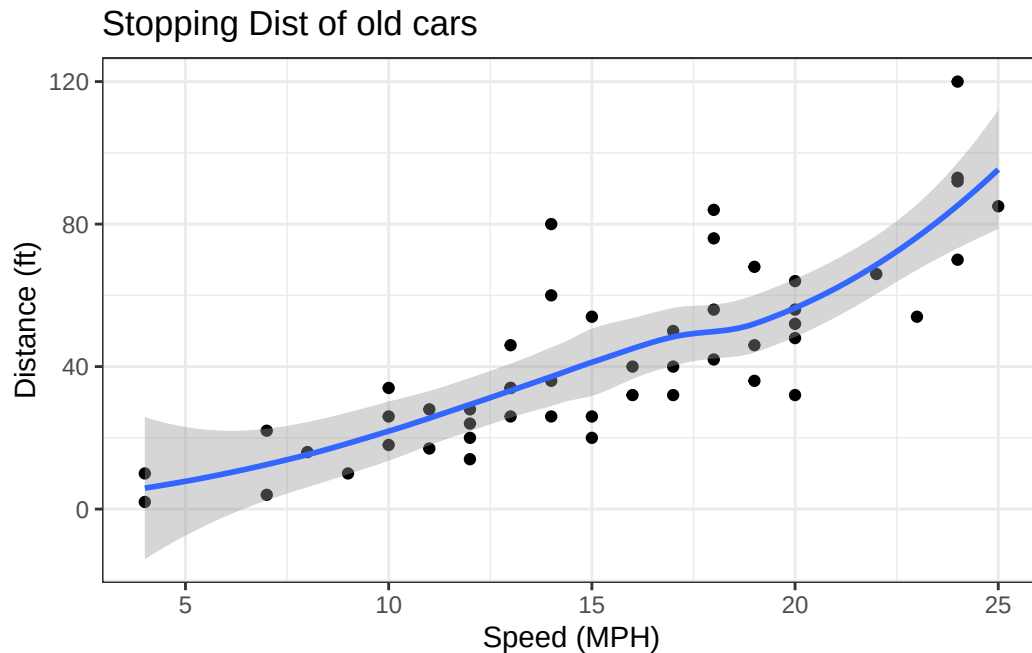
```
ggplot(cars) +  
  aes(x=speed, y=dist) +  
  geom_point() +  
  geom_smooth()
```

``geom_smooth()`` using `method = 'loess'` and `formula = 'y ~ x'`



```
ggplot(cars) +  
  aes(x=speed, y=dist) +  
  geom_point() +  
  geom_smooth() +  
  theme_bw() +  
  labs(title="Stopping Dist of old cars",  
        x="Speed (MPH)",  
        y="Distance (ft)")
```

``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



Q. Can you make the 'geom_smooth()' function produce a linear straight line fit to the data and turn off the "gray" error region.

Gene Expression Figure

Import the data to plot

```
url <- "https://bioboot.github.io/bimm143_S20/class-material/up_down_expression.txt"
genes <- read.delim(url)
head(genes)
```

	Gene	Condition1	Condition2	State
1	A4GNT	-3.6808610	-3.4401355	unchanging
2	AAAS	4.5479580	4.3864126	unchanging
3	AASDH	3.7190695	3.4787276	unchanging
4	AATF	5.0784720	5.0151916	unchanging
5	AATK	0.4711421	0.5598642	unchanging
6	AB015752.4	-3.6808610	-3.5921390	unchanging

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```
[5125] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
[5137] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[5149] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[5161] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[5173] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[5185] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

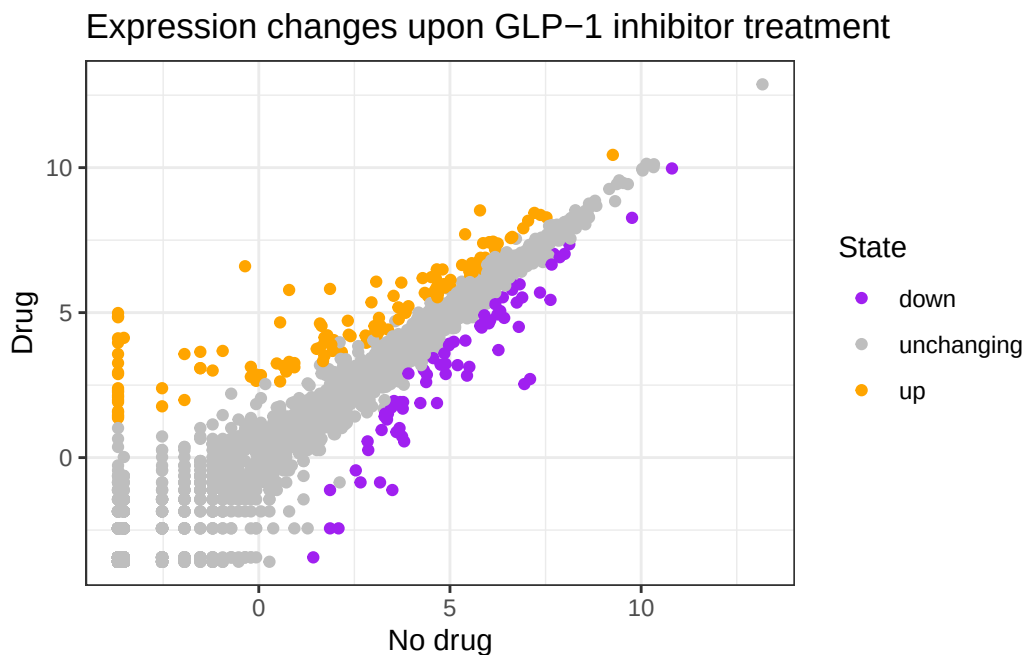
A useful new function in this context is the `table()` function:

```
table(genes$State)
```

```
down  unchanging      up
   72      4997     127
```

My first plot attempt

```
ggplot(genes) +
  aes(Condition1, Condition2, col=State) +
  geom_point() +
  scale_color_manual(values=c("purple","gray","orange")) +
  theme_bw() +
  labs(title="Expression changes upon GLP-1 inhibitor treatment", x="No drug", y="Drug")
```



Going further

Here we read the famous gapminder dataset:

```
# File location online
url <- "https://raw.githubusercontent.com/jennybc/gapminder/master/inst/extdata/gapminder.tsv"

gapminder <- read.delim(url)
head(gapminder)
```

	country	continent	year	lifeExp	pop	gdpPercap
1	Afghanistan	Asia	1952	28.801	8425333	779.4453
2	Afghanistan	Asia	1957	30.332	9240934	820.8530
3	Afghanistan	Asia	1962	31.997	10267083	853.1007
4	Afghanistan	Asia	1967	34.020	11537966	836.1971
5	Afghanistan	Asia	1972	36.088	13079460	739.9811
6	Afghanistan	Asia	1977	38.438	14880372	786.1134

Q. How many entries (i.e. rows) are in this dataset?

```
nrow(gapminder)
```

```
[1] 1704
```

Q. How many different “country” entries are there in this dataset?

```
length(table(gapminder$country))
```

```
[1] 142
```

```
length(unique(gapminder$country))
```

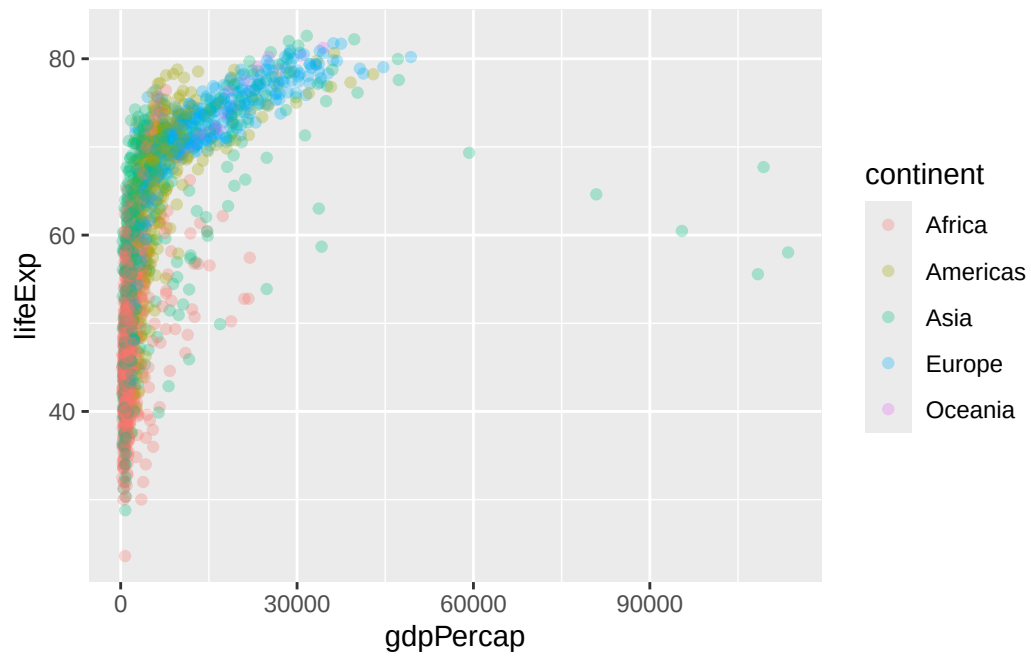
```
[1] 142
```

Let’s make our first plot of the entire dataset:

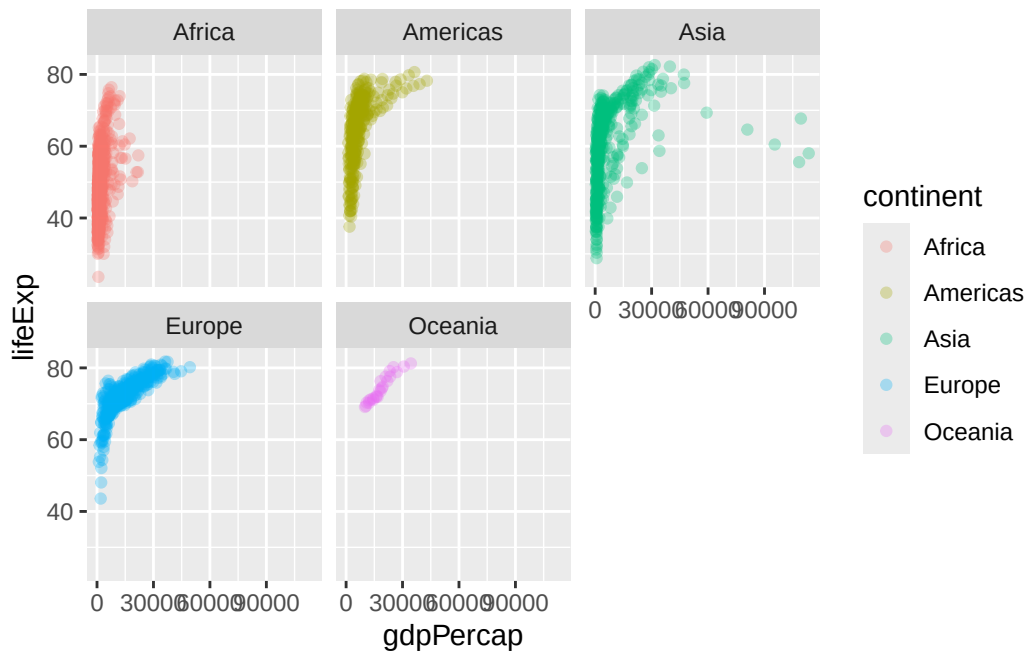
Plot of “gdpPercap” vs “lifeExp” colored by “continent”

```
p <- ggplot(gapminder) +  
  aes(x=gdpPercap, y=lifeExp, col=continent) +  
  geom_point(alpha=0.3)
```

p



```
p +  
  facet_wrap(~gapminder$continent)
```



Make a plot for years 1977 and 2007 only (not all the years in the dataset).

Q. First use the **dplyr** package and the **filter()** function from that package to extract the rows from the year 2007.

```
library(dplyr)
```

```
g07 <- filter(gapminder, year == 2007)
g77 <- filter(gapminder, year == 1977)
g <- filter(gapminder, year == 2007 | year == 1977)
```

```
ggplot(g) +
  aes(gdpPercap, lifeExp, col=continent, size=pop) +
  geom_point() +
  facet_wrap(~year)
```



Q. Make a histogram of lifeExp colored by continent (use `fill=continent` or `col=continent`)

Q. Make a histogram of lifeExp faceted by continent

```
ggplot(gapminder) +
  aes(lifeExp) +
  geom_histogram()
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.

